

408 真题·虚拟内存：一次访存的全流程

标记

	真题考过	课本上有	写法
●	考过	有	展开：哪几年问过、怎么问的、变体、结论
◇	没考过	没有	标记是菱形；内容放灰底框，标「课本外补充」，从真机补
—	没考过	有	流程经过，但不产生新内容

线路总表

步	环节	哪个组件	标记
第一段·快路径（TLB 命中）			
1	CPU 产生虚拟地址	硬件	●
2	MMU 用虚页号查 TLB	硬件	●
3	命中，取出页框号	硬件	—
4	拼出物理地址	硬件	●
5	Cache 查表	硬件	●
6	数据回 CPU	硬件	—
第二段·慢路径（TLB 未命中，页在内存）			
7	读 CR3 拿一级页表基址	硬件	●
8	硬件逐级遍历页表	硬件	●
9	每级查存在位与权限位	硬件	●
10	末级权限检查（含 NX）	硬件	●
11	填充 TLB	硬件	◇
12	回到第 4 步	—	—
第三段·缺页路径（页不在，或权限不符）			
13	出错地址写进 CR2	硬件	◇
14	抛 #PF，附 error code	硬件	◇
15	用 IDTR 找处理程序入口	硬件	●
16	保存断点	硬件	●
17	保存通用寄存器	软件	●
18	切到内核态	硬件	●
19	内核读 CR2 与 error code	软件	◇
20	查 VMA 判断地址合法性	软件	◇
21	不合法 → SIGSEGV	软件	●
22	合法 → 分配页框，可能触发置换	软件	●
23	文件映射 → 走 page cache	软件	●
24	发磁盘 I/O，进程让出 CPU	软件	●
25	磁盘中断，数据到位	硬件	◇
26	填页表项	软件	●
27	反向映射更新	软件	◇
28	多核 → TLB shutdown	硬+软	◇
29	返回用户态，重新执行本条指令	—	●
30	重走第 1 步，这次 TLB 命中	—	●

30 步里 ● 19 步考过真题，65 道全落在这 19 步上。◇ 只剩 8 步——课本和真题都没写到的地方，合起来只占几页。它们全部集中在慢路径和缺页路径：快路径真题标满了，越往异常路径走，越稀。

第一段·快路径

课本外补充 这一段是绝大多数访存的结局：一次访存里，TLB 命中的比例通常在 99% 以上。整条慢路径和缺页路径加起来，是为了让这一段看起来像「内存直接就能访问」。

1 CPU 产生虚拟地址

硬件 ●

指令里的地址经寻址方式算出偏移，加上段基址／基址寄存器，得到**虚拟地址**。这是全线的起点，也是真题里最大的一块。

量	求法
页内偏移位数 k	$k = \log_2(\text{页大小})$
虚页号位数	虚拟地址位数 $-k$
页框号位数	物理地址位数 $-k$
虚拟地址位数	$\log_2(\text{虚拟地址空间大小})$
页表项数	$2^{\text{虚页号位数}}$
一页能放的页表项数	页大小 $\div$ 页表项大小

先定页内偏移，剩下的全归页号。地址的每一位都有归属，这一步定了，后面所有量都能推出来。

单位易错：2022-15 的页表里虚页号那列末尾没写 H，是十进制，查表前必须先转换。2026-19 和 2024-45 也都在这一步设过换算。

这个地址空间的性质

- 每个进程都有自己独立的虚拟地址空间；`malloc()` 返回的是虚拟地址；数据段和代码段可以有不同的访问权限（2023-28）
- 虚拟地址的大小由虚拟内存管理机制和操作系统定义，不是由主存、硬盘的容量决定（2023-28）
- 虚拟存储只能建立在**离散分配**基础上（请求分页／请求分段／请求段页式），不是连续分配（2012-25）
- 虚拟存储的容量由 **CPU 的寻址范围** 决定，不是只受外存或内存容量限制（2012-25）
- 逻辑地址形成于**链接阶段**——编译产生的是各模块都从 0 开始、仅相对于本模块的地址；链接器完成重定位后，才形成整个程序的完整逻辑地址空间（2011-30）

出处：2011-30、2011-44、2012-25、2013-16、2013-46、2015-46、2019-31、2020-46、2021-44、2022-15、2023-28、2024-17、2024-45、2026-19

课本外补充 x86-64 的虚拟地址是 **48 位**，但寄存器是 64 位——高 16 位必须是第 47 位的符号扩展，这种地址叫「规范地址」。不满足这个条件的地址，CPU 直接判为非法，根本走不到 MMU。这也解释了为什么 64 位机器的虚拟地址空间不是 2^{64} 而是 2^{48} ：多出来的位不是给你用的，是留给将来扩的。后来确实扩了——第五级分页把地址加到 57 位。

2 MMU 用虚页号查 TLB

硬件 ●

TLB 就是页表的 **Cache**。它的索引方式和 Cache 完全一样，按处理 Cache 的方式求解即可。

量	求法
组数	表项数 ÷ 路数
组号位数	$\log_2(\text{组数})$
TLB 标记位数	虚页号位数 − 组号位数
组号映射	TLB 组号 = 虚页号 mod 组数

标记字段里放的是虚页号的高位部分——2016-45 明确问过这一点。

命中关系铁律 (2010-17): Cache 命中 $\Rightarrow$ 页必命中; TLB 命中 $\Rightarrow$ 页必命中; TLB 未命中 $\Rightarrow$ 页可能命中 (快表只是页表的部分副本); **不可能**发生 TLB 命中而页未命中。

用什么实现 (2020-15): TLB 用相联存储器 (也可用 SRAM), Cache 用 SRAM, **都不能用 DRAM**。

MMU 在这一步同时检测三件事 (2024-18): 访问越权、页面缺失、TLB 缺失。**Cache 缺失不在其中**——它发生在这条线路的更后面, 第 5 步。

加快虚实地址转换 (2014-28): 增大 TLB 容量 $\checkmark$ 、让页表常驻内存 $\checkmark$ 、增大交换区 $\times$ 。

课本外补充 TLB 是分层的。真实的 CPU 里通常有一级数据 TLB、一级指令 TLB、加一个二级统一 TLB, 项数逐级放大。关键指标是 **TLB reach**——能同时覆盖多大的地址范围, 等于「项数 $\times$ 页大小」。4KB 页下, 哪怕二级 TLB 有上千项, 覆盖范围也只有几 MB。这就是第 8 步要讲的大页存在的真正理由。

TLB 项里还有进程标签 (ASID / PCID), 有了它, 换进程时旧条目不必全部作废——第 7 步会用到。

3 命中, 取出页框号

硬件 —

命中就直接拿到物理页框号, **整条慢路径一步都不走**。这是 TLB 存在的全部意义: 把第 7 到第 11 步压缩成一步。

4 拼出物理地址

硬件 ●

物理地址 = 页框号 || 页内偏移

是**拼接**, 不是相加。因为页大小是 2 的幂, 拼接恰好等价于「页框号 $\times$ 页大小 + 页内偏移」。

关键推论: 页大小 = $2^k \Rightarrow$ 虚拟地址与物理地址的**低 k 位完全相同**。这条直接推出第 5 步的全部内容。

出处: 2013-46、2020-46、2024-45

5 Cache 查表

硬件 ●

解题依据: 页大小 = $2^k \Rightarrow$ 虚实地址低 k 位相同。由此:

- 页内偏移那部分, 可以直接用**虚拟地址的位**做 Cache 索引
- 页号那部分, 必须翻译成物理页框号之后才能用

2019-46 就是这条: 页大小 4KB $\Rightarrow$ 低 12 位相同 $\Rightarrow$ call 指令虚拟地址的低 12 位即物理地址低 12 位 $\Rightarrow$ 取其中第 6~9 位得 Cache 组号。

两个层次的对照 (2024-16、2016-45)

	Cache--主存	主存--外存
交换单位	主存块	页
替换算法	硬件实现	软件实现
写策略	可直写、可回写	总是回写
映射方式	直接／组相联／全相联	页式映射

主存--外存若采用直接映射，替换算法就完全失效了——2024-16 的 D 选项错误的正是这里。
缺失代价 (2016-45): 缺页的开销远大于 Cache 缺失——缺页要访问磁盘，Cache 缺失只需访问主存；也可以说：缺页由软件处理，Cache 缺失由硬件处理。

课本外补充 真实 CPU 的 L1 Cache 用的是 **VIPT**: 组号用虚拟地址的位 (译码早, 不必等 MMU 翻译完), 标记用物理地址的位 (翻译完成后才能比对)。
它能成立的前提正是「索引位必须全部落在页内偏移里」。由此得到一条硬约束:

$$\text{Cache 容量} \div \text{路数} \leq \text{页大小}$$

4KB 页下, 32KB 八路正好卡在边界上。这不是巧合——页大小和 Cache 参数是互相约束着设计的。
L2 及更高层不用受这个约束, 因为它们本来就要等 MMU 的结果, 用物理地址索引 (PIPT) 更省事, 也避免了同一个物理页被映射到两个虚拟地址时的别名问题。

6 数据回 CPU

硬件 —

命中就结束, 指令继续往下走, 一次访存在这里画句号。慢路径和缺页路径存在的意义, 就是让这一步的成功率尽可能高——它们是例外处理, 不是常规流程。

第二段·慢路径

课本外补充 TLB 里没有这一项，但页其实在内存里（只是没被缓存进 TLB）。这一段纯粹是硬件在读页表——操作系统完全不知情。
课本里的页表结构和多级页表，讲的都是这一段。

7 读 CR3 拿一级页表基址 硬件 ●

页表基址寄存器里存的是当前进程一级页表的起始物理地址。课本叫 PTBR / PDBR，x86 上叫 CR3。
为什么必须是物理地址：因为访问页表本身如果还要再做一次地址翻译，就会无限递归。所以这一步不能再翻译，必须直接用物理地址。
进程切换时更新它，线程切换时不更新（2024-25、2025-23）——线程共享地址空间。而内核中断向量表基址寄存器是系统级的，操作系统启动时设置一次，任何切换都不更新（2018-45）。
出处：2021-29、2018-45

课本外补充 换 CR3 有个真题从不提、真实机器上却很贵的副作用：TLB 里属于旧进程的条目全部作废。地址空间都换了，缓存的映射当然不能再信。新进程一开始必然是一片 TLB 缺失——这是进程切换比线程切换昂贵得多的主要原因之一。
现代 CPU 用 PCID 给 TLB 条目打上进程标签来缓解，但「切换地址空间 ⇒ TLB 失效」这个基本事实没变。

8 硬件逐级遍历页表 硬件 ●

为什么分级：减少页表占用的连续内存空间。把一张大页表分级，每张控制在一页之内。
它不加快地址变换（反而多查几次）、不减少缺页中断、不减少页表项所占字节数。2014-32 的四个选项就是照这个设的。

问的是什么	怎么算
页目录表项数	逻辑地址空间页数 ÷ 一页能放的页表项数
页目录占几页	页目录项数 × 项大小 ÷ 页大小
页表占几页	页表项总数 × 项大小 ÷ 页大小
页表项物理地址	页表起始物理地址 + 页号 × 页表项大小
最末级页表张数	各级索引容量的乘积

2010-29：逻辑地址空间 2^{16} 页，每页放 $2^{10}/2 = 2^9$ 个页表项，故 $2^{16}/2^9 = 2^7 = 128$ 。
2015-46：页目录 = $2^{10} \times 4B/4KB = 1$ 页；页表 = $2^{20} \times 4B/4KB = 1024$ 页。
2026-28：每级索引 n 位 ⇒ 每张页表 2^n 项；最末级页表的张数 = 各级索引容量的乘积。
出处：2010-29、2014-32、2015-46、2020-46、2021-29、2026-28

课本外补充 课本里的二级页表——10|10|12、每张表 1024 项、页表项 4 字节——是 32 位 x86 的真实结构。今天的机器不是这样：

项目	32 位（考试模型）	x86-64（真实）
虚拟地址	32 位	48 位
切分	10 10 12	9 9 9 9 12
级数	2 级	4 级
每级项数	1024	512
页表项大小	4 B	8 B

关键在最后两行。为什么每级是 512 项？因为 $512 \times 8\text{B} = 4096\text{B} = \text{恰好一页}$ 。真实机器要求每张页表正好占一页，这样分配和定位都最简单——于是「每级多少项」不是随便定的，**它被页大小和页表项大小同时锁死**。
这正是课本里那条公式

一页能放的页表项数 = 页大小 ÷ 页表项大小

的来历。它不是一道算术练习题，而是分级的物理约束。
2026-28 那道题是照着 x86-64 改的。它给的 9|9|9|12 正是真实的 9 位索引，只是把 4 级砍成 3 级，多出来的 9 位塞进了那个「补充位 (25)」。
走这一趟要访存几次？四级就是四次——每读一级页表项都要访存。但页表项自己也在 Cache 里，所以实际是四次 Cache 访问，多数命中。这就是为什么 TLB 缺失没有想象中那么贵，也是为什么**大页有用**：2MB 大页把四级压成两级，页表项少、占的 TLB 条目少、遍历级数也少。
课本说「多级页表不能降低平均访存时间」是正确的——真实机器承认这个代价，然后用大页去补。

9 每级查存在位与权限位

硬件 ●

10 末级权限检查（含 NX）

硬件 ●

这就是 2024-18 说的「访问越权」和「页面缺失」被检测的地方。页表项里几个关键位：

位	名	作用
0	P	存在位，就是课本的 有效位 。 $P = 0$ 时其余位被硬件忽略，全部交给操作系统自用——Linux 就是在这里存该页在交换分区里的位置。
1	R/W	读写权限。「越权」的来源。
2	U/S	用户态／内核态。特权级保护的来源。
5	A	Accessed， 硬件在访问时自动置 1。
6	D	Dirty， 硬件在写入时自动置 1。
7	PS	大页标志。
63	NX	不可执行位。

第 5、6 位是第 22 步的伏笔，到那里再说。

11 填充 TLB

硬件 ◇

课本外补充 走到这里，硬件已经拿到了有效的页表项，顺手把它写进 TLB，下次同样的虚页号就能在第 2 步命中。
哪个组件来填，两种架构不一样：

- **x86**：TLB 未命中由**硬件**自动遍历页表并回填，软件完全不知情，只是慢一点。

- **MIPS、SPARC** 等：TLB 未命中会触发一个专门的异常，由**操作系统**用软件遍历页表、填 TLB。

课本里「只有访存指令会缺页」「一次缺页要多访问一次 TLB」（2014-45）说的是缺页那一级；TLB 未命中本身要不要陷入操作系统，取决于架构。408 不区分这两者。

12 回到第 4 步

拿到页框号，回到第 4 步拼物理地址。这一段结束。

第三段·缺页路径

课本外补充 页不在了，或者权限不符。这一段是硬件和软件的交界——硬件负责发现并抛出，操作系统负责处理。

2026 年连号考的四道题（特权指令、中断响应中硬件完成的工作、内核模式执行的操作、虚拟存储器的指令执行），全部落在这一段。所谓「突然考 MMU」，实际上是考试的重心在往这条线路上移。

13 出错地址写进 CR2

硬件 ◇

14 抛 #PF, 附 error code

硬件 ◇

课本外补充 硬件发现异常后要做两件准备工作，都在这两步：

CR2：把出错的虚拟地址写进去。操作系统想知道是哪个地址出错，读它就行——不用去反推指令。

error code：把出错原因编码好一起压栈。哪几位分别是：这次访问是读还是写、发生在用户态还是内核态、是页根本不存在还是权限不够。

这个编码很关键：「页不存在」和「权限不够」在硬件看来是同一类事件，在操作系统看来是两件完全不同的事——前者要调入页面，后者要报段错误。判据就是 error code。

15 用 IDTR 找处理程序入口

硬件 ●

硬件拿着中断／异常号去查**中断描述符表**，表的基址存在 **IDTR**（中断向量表基址寄存器）里。缺页是 14 号。

2018-45 考过这个寄存器——问的是「进程切换时要不要更新它」，答案是不更新：它是系统级的，操作系统启动时设置一次，所有进程共用一张表。

16 保存断点

硬件 ●

17 保存通用寄存器

软件 ●

这两步紧挨着，却是两个不同的执行者。2026-22 那道题把四件事并列成选项，问「必须由硬件完成的是」，答案是**保存断点**：

- **保存断点由硬件负责**——把 CS:RIP、RFLAGS（缺页还要压 SS:RSP）压栈。因为中断是随时来的，「保存」这个动作本身如果由软件指令完成，它自己就可能被打断，那就永远保存不完。
- **保存通用寄存器由软件负责**——硬件不知道这个进程用了哪些寄存器。

开中断也是软件负责的，通过指令实现，用于允许或禁止中断嵌套。

18 切到内核态

硬件 ●

硬件把特权级从用户态切到内核态，处理器开始执行操作系统代码。

这一步加它前面两步，是 2026-21、2026-23 的地盘：

特权指令（2026-21）：只能在核态下执行的指令。I/O 指令、关中断指令都在其中。而**中断返回指令不是特权指令**——它就是那道题设的那个选项。

内核模式执行的操作（2026-23）：编译程序、链接程序、命令解释程序都跑在用户态；**装入程序**要把可执行文件加载进内存、启动进程，涉及内存分配和进程创建，由内核在核态执行。

19 内核读 CR2 与 error code

软件 ◇

20 查 VMA 判断地址合法性

软件 ◇

课本外补充 操作系统拿到出错地址和原因，第一件事是**判断这个地址该不该给你**。Linux 把每个进程的地址空间切成一段一段，叫 VMA（虚拟内存区域），用**红黑树**组织，查找是 $O(\log n)$ 。**红黑树这个考点，18 年真题一次都没出现过——它就在这里**。它不是在数据结构的抽象世界里待着，它是缺页处理这条线路上的必经一步。判完合法性才决定下一步：合法就往下走第 22 步，不合法就是段错误。

21 不合法 → SIGSEGV

软件 ●

课本上「越界／越权／段缺失」三种异常的分类，落到真实机器里就是这一步的三个分支。2016-28 问的越界异常，判据是段表项里存的段长——段内地址 > 段长就抛出。

22 合法 → 分配页框，可能触发置换

软件 ●

全卷最大的一团。缺页处理的标准流程（2011-28、2013-30）：

1. 在内存中找空闲页框并分配（可能涉及页面置换）
2. 调用设备驱动程序做磁盘 I/O，把缺失的页调入内存
3. 修改页表：存在位置 1，填入物理页框号

第一步不一定包含淘汰（2022-29）——内存里还有空闲页框时直接分配就行。

页面置换算法

- **Belady 异常**（2014-30）：只有 **FIFO** 会产生——页框数增加，缺页次数反而增加。LRU 和 OPT 都不会。
- **LRU 手算**（2015-27）：从访问序列**从后往前**数，数到第 n 个（ $n =$ 页框数）**不同的**页号，停下时那个页号就是被淘汰的。
- **改进型 CLOCK**（2016-26）：按（访问位 A，修改位 M）把页分四类，淘汰顺序 $(0,0) \rightarrow (0,1) \rightarrow (1,0) \rightarrow (1,1)$ 。先找最近没访问过的；同是没访问过，优先淘汰没修改过的。
- **基本 CLOCK**（2010-46）：指针扫过页框，使用位为 1 就清零并继续，为 0 就替换该页；新页装入后把使用位置 1。

置换次数 $\neq$ 缺页次数（2019-29）——进程刚启动时页框是空的，前几次缺页只是「装入」，不触发置换。问「置换次数」时必须扣掉这部分。

页框分配与驻留集

- **策略组合 (2015-30)**: 「固定分配 + 全局置换」不能组合使用——固定分配意味着每个进程的页框数不变, 而全局置换允许从别的进程那里抢页框, 自相矛盾。可组合的三种: 可变分配 + 全局置换、可变分配 + 局部置换、固定分配 + 局部置换。
- **工作集 (2016-29)**: 最近 k 次内存访问所访问过的页面集合, 重复的只算一次。
- **抖动 (2011-29)**: 刚被换出的页很快又要访问, 于是又换出别的页, 频繁置换, 大部分时间花在换页上。**有效措施**: 撤销部分进程 (减少所需页面数)。无效: 增加对换区容量、提高用户进程优先级。
- **最少页框数取决于 (2025-27)**: 指令系统的寻址方式——支持间接寻址或跨页访问就需要更多页框。不是代码段长度, 不是虚拟/物理地址空间大小。
- **页框数与位图 (2023-25)**: 页框数 = 物理内存 $\div$ 页大小; 位图大小 = 页框数 $\div$ 8 字节。

影响到哪一层

- **影响缺页率 (2022-30)**: 置换算法 $\checkmark$ 、工作集大小 $\checkmark$ 、进程数量 $\checkmark$ 。**不影响**: 页缓冲队列的长度 $\times$ ——它只影响页面置换的速度。
- **影响平均访存时间 (2020-28)**: 缺页率、磁盘读写时间、内存访问时间、执行缺页处理程序的 CPU 时间——四项全选。
- **能降低平均访存时间 (2026-29)**: TLB $\checkmark$ 、工作集 $\checkmark$ 、页表缓冲队列 $\checkmark$ 。**多级页表 $\times$** ——它省的是空间, 反而增加地址转换步数。

课本外补充 A 位和 D 位为什么可用。第 9 步那张表里的第 5、6 位, 是硬件在访问时顺手置的——软件零成本。

这解释了两件事:

一、为什么 **LRU** 理论上最好、实际只能用 **CLOCK** 近似。精确 LRU 要记录「上次访问的时间」, 硬件不提供这个。而 A 位是现成的。

二、为什么改进型 **CLOCK** 优先淘汰「没修改过」的。D=0 意味着这一页和磁盘上一致, 不必写回磁盘, 淘汰代价小得多。所以 (0,0) 排在 (0,1) 前面。

真实 Linux 用的是近似 LRU 双链: 每个内存区维护 active 和 inactive 两条链表, 页在两条链之间流动, 回收时从 inactive 尾部取。这是 CLOCK 思想的一个变体, 同样只依赖 A 位。

回收页框不是只在缺页时才发生。内核有专门的后台线程 (kswapd) 在内存低于水位线时就开始回收, 不等真的缺页。所以第 22 步触发置换的时候, 往往已经有现成的空闲页框了——这正是 2022-29 说的「不一定包含淘汰」。

23 文件映射 $\rightarrow$ 走 page cache

软件 ●

2025-30 考的内存映射文件: 把文件直接映射进进程的虚拟地址空间, 可实现进程间通信 (多个进程映射同一文件, 共享同一份物理页面)。注意那道题的错误选项「实现了页面到磁盘块的映射」方向说反了——是「文件页 $\rightarrow$ 虚拟页」, 不是反过来。

2023-30 的共享数据是同一件事的另一面: 两个进程共享 data, 各自页号 p_1 、 p_2 可以不相等; 但页框号 $f_1 = f_2$ 一定相等。

课本外补充 真实内核把这类页叫**文件页**, 统一放在 page cache 里。同一个文件被几个进程映射、或者被 read() 读, 用的都是同一份缓存——这是「页框号一定相等」的物理来源。与之相对的是**匿名页** (堆、栈), 没有文件后备, 换出时只能写进交换分区。

24 发磁盘 I/O, 进程让出 CPU

软件 ●

缺页的代价就是这一步：磁盘访问是毫秒级，内存是纳秒级，差六个数量级。2016-45 说的「缺页开销远大于 Cache 缺失」就是比在这里。

等待期间进程被挂起，CPU 交给别人——所以一次缺页的代价不只是磁盘时间，还有一次上下文切换。

25 磁盘中断，数据到位

硬件 ◇

课本外补充 磁盘控制器发中断，内核的 I/O 完成处理程序接管，唤醒刚才睡眠的进程，把它放回就绪队列。

这一次中断走的是第 15 步那套机制——同一个 IDTR，只是向量号不同。

26 填页表项

软件 ●

缺页处理三步的最后一步：存在位置 1，填入物理页框号，设置权限位。

页表项的内容是操作系统在运行时动态设置的（2026-24）——不是编译器决定的。编译器只产生虚拟地址，页表长什么样、哪一页在内存里、映射到哪个页框，全是内核运行时的决定。

这就是 2026-24 那道题的 B 选项错误的地方，也是它 A、C 两个选项错误的同一个来源：**地址转换由硬件负责、缺页处理由软件负责**，分界线就在这一段线路上——第 13 到第 18 步是硬件的活，第 19 到第 26 步是操作系统的活。

27 反向映射更新

软件 ◇

课本外补充 一个物理页框可能被多个进程、多个虚拟地址映射（第 23 步的共享就是）。所以要换出它的时候，必须能找到**所有**映射了它的页表项，逐个改掉——内核为此在页框这一侧维护反向映射结构。

没有这个结构，共享页就没法安全换出。

28 多核 → TLB shutdown

硬+软 ◇

课本外补充 页表改了，可是 TLB 里还留着旧映射——而且**别的 CPU 核上的 TLB 也留着**。本核可以自己作废（一条 `invlpg` 指令），别的核作废不了，只能给它们发**处理器间中断**（IPI），让它们各自作废。这是多核机器上修改页表的隐藏成本。单核时代没有这个问题，所以课本里也不会提。

29 返回用户态，重新执行本条指令

— ●

返回到发生缺页的那条指令重新执行，不是下一条（2019-14）。这是最经典的错误选项。

为什么必须重新执行：出错的指令还没有完成——它连地址都没翻译成功，取数据更谈不上。程序计数器停在这条指令上，处理好之后必须重来一遍。

一次缺页 = 多访问一次 TLB（2014-45）：第一次访问某页要查一次 TLB，触发缺页；处理完重新执行这条指令，又要查一次 TLB。所以访问 n 次数据、其中一次缺页，TLB 访问次数是 $n + 1$ 。

只有访存指令会缺页（2014-45）——加法、移位、寄存器自增这类运算指令都不会，它们不访问内存。

30 重走第 1 步，这次 TLB 命中



重新执行时从头再做一遍，这次第 2 步命中——除非中间被 shutdown 掉了。一次缺页的处理到此结束。

附·另一条路：分段与段页式

前面铺的是页式这条线路。分段是另一种地址转换方式，在第 8 步那个位置换一套做法。

最大段长 (2009-27)：逻辑地址 = 段号 + 段内位移。地址 m 位、段号 n 位 $\Rightarrow$ 位移 $m - n$ 位 $\Rightarrow$ 最大段长 = 2^{m-n} 。(32 位地址、段号 8 位 $\Rightarrow$ 段内位移 24 位 $\Rightarrow$ 最大段长 2^{24} B)

越界异常 (2016-28)：段表项里存了段长。段内地址 $>$ 段长 $\Rightarrow$ 越界异常。注意区分三种异常：段缺失、越权、越界。

共享段 (2019-28)：

- 物理内存中只保存一份段的内容 ✓
- 共享段的段表项物理地址唯一 ✓
- 所有进程都不再使用时才能回收 ✓
- 不同进程中的段号可以不同 ✗

段页式则是先分段、段内再分页，等于把两套机制合起来用：先用段表拿到该段的页表基址，剩下和第 7 步之后完全一样。